Email Ops Agent

An email-to-action agent where a person approves every change. Portfolio case study.

The problem

Shared inboxes run on people re-typing the same information into a CRM and a task tracker. It is slow and inconsistent, and the classic failure is the duplicate: a follow-up filed as a new issue because nobody remembered the first. An AI that "handles email" on its own is the thing a careful client will not accept: it writes to production systems unsupervised, cannot explain itself, and sends customer email to a third party.

What it does

email > classify > extract > match existing records > propose > a person approves, edits or rejects > execute via API > audit log. A sales inquiry becomes a Lead in EspoCRM, a support request a work item in Plane. A known sender or a follow-up becomes an update, not a duplicate. Anything that is not a customer message is escalated to a person. Nothing is written until someone approves, and every step is kept in an append-only audit trail.

Design choices that matter

- **Approval is enforced in code.** The decision is recorded first; a rejected or unreviewed proposal never reaches the execution code. A reviewer can correct the proposed text first; the audit trail keeps both the model's version and the human's.
- **Runs on the client's own infrastructure.** Classification and extraction use a self-hosted open-weight model (Mistral 7B via Ollama), so email content stays on-prem. Swappable for Claude with one setting.
- **Narrow interfaces, not one big agent.** The model and each target system sit behind a small interface; adding Zammad or ERPNext means writing an adapter, not changing the pipeline. Each stage is logged and inspectable.
- **Matching fitted to each system.** CRM by exact email, then fuzzy company name. Plane has no search, so the agent tags each issue with the sender and matches follow-ups against that sender's issues only.

Results from running it on real systems

Six scripted emails in three acts, a real EspoCRM and a real Plane project, a person approving each proposal. The first run exposed problems; I fixed them and ran again.

	First run (5 Oct)	Second run (15 Oct)
Emails processed / executed	6 / 5 of 5 approved	6 / 6 of 6 approved
Follow-ups updated, not duplicated	2 of 2	2 of 2
Non-customer emails routed to a person	1 of 2	2 of 2
Junk reaching the CRM	0 (rejected at review)	0 (never proposed)
Plane issue title	the sender's name	the email's subject
Lead description	the word "sales_inquiry"	a summary of the request

Read this honestly. It is one run on the same six emails the fixes were written against, so it shows the specific failures are fixed, not an accuracy rate. Five fresh emails the prompts had not been tuned on were all classified as expected, but that is five. In the first run the model filed recruiter spam as a sales inquiry; a reviewer rejected it and nothing reached the CRM, which is the point of keeping a person in the loop. Self-reported model confidence is still not a reliable signal.

Built like a project that will be maintained

- Tested (129 tests), type-checked and linted, with CI on every change. Schema changes go through migrations, with a test that fails if a model change has none.
- Login with hashed passwords and the reviewer recorded on every decision; protection against cross-site request forgery and page framing; a readiness check that reports whether the model is actually loaded.
- Findings from live runs fixed at the root: failed emails are retried instead of lost, a proposal cannot be decided twice, and slow polls run in the background with visible progress.

What it does not do yet

It reads from an email simulator, not a real inbox. It is slow on CPU (about 3 to 4 minutes per email), free-text extraction is the main open risk, login is HTTP Basic (needs TLS outside localhost, no lockout, no roles), and automatic execution without approval is deliberately out of scope until there is real accuracy data.

If this were a client engagement

Swap the simulator for a sandboxed real inbox, add one or two of the client's own systems as adapters, and run approve-everything for a few weeks. That produces the accuracy data needed to decide, per action type, whether anything can safely skip review.

Python, FastAPI, SQLAlchemy, Alembic, PostgreSQL, Ollama (Mistral 7B), EspoCRM, Plane, Docker Compose, GitHub Actions. | Source and full write-up: github.com/mirzacanovic/email-ops-agent